



Mobile and Embedded Computing

Lecture 1: Course orientation · Platforms & the device landscape

Dinu-Ştefan Rusu

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro

Microsoft Teams: course channel

rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with



The 12 lectures

01	The mobile & embedded landscape; native vs cross-platform; Git & GitHub Today	07	Authentication; OAuth providers; Firebase App Check
02	Compiled vs interpreted languages; null safety; Dart & Kotlin; Flutter intro	08	WebSockets; runtime permissions; routing
03	Agent-assisted coding; Flutter widgets; async, isolates vs goroutines	09	AI on-device and connecting to LLMs
04	Debugging; Stateless vs Stateful; introduction to state management	10	Offline-first: local data, sync and conflict resolution
05	Server vs client-side execution; serverless vs VPS; Go backends & Firebase	11	Performance & energy; embedded with TinyGo (sensors → MQTT → your app)
06	Packages & feature flags; GraphQL & REST; observability and Crashlytics	12	gRPC; platform channels & FFI; Kotlin Multiplatform

Grading

3_p

Final exam

in the exam session

3_p

**Laboratory +
assignments**

individual

1.5_p

Lab test

individual

1.5_p

Project

teams of max 3

1_p

Concept presentation

teams of max 3

You pass with a total of ≥ 5 out of 10.

Concept presentations

Pick a topic from the shared Excel table; topics are taken in the order they are claimed

Prepare slides that cover ~20 minutes of presentation

If it fits the topic, include a demo, inside the same 20 minutes

Every team member presents their part

Upload all materials to Moodle in the designated space

You present from your own laptop, at the projector

Teams of at most 3 students

Speaking time

20 min ÷ team size

team of 2 → ~10 min each

team of 3 → ~7 min each

TODAY

What you should leave with



The device spectrum

The range of devices this course targets, and how their constraints differ by orders of magnitude



Four build strategies

The four ways to build a mobile app, and the trade-offs of each



Why Flutter

Why this course uses Flutter, and what that choice costs



A working Git setup

A GitHub account, and the branch → PR → review workflow the project grades

Git and the services that host it

Git is a version control system: it tracks every change ever made to your source code

It lets multiple developers work on the same codebase without overwriting each other

Git is a tool on your machine. GitHub, GitLab and Bitbucket are services that host Git repositories and add collaboration on top: reviews, issues, CI

This course uses GitHub



one Git

GitHub · GitLab · Bitbucket

many hosting services

Create a GitHub account: it is mandatory

Sign up, free: github.com/signup

Your commits are part of your grade: labs, assignments and the team project

Every resource for this course lives in one repository:

```
github.com/rusudinu/mobile-and-embedded-computing
```



Repositories and branches

A repository holds all the files of your project, and every revision they've ever had

It always has at least one branch (main)

A repository is a Git concept, not a GitHub one. GitHub only hosts it

Branches let you build features, fix bugs and experiment without stepping on your teammates' work

A branch is always created from an existing branch



`main` → `feature/...`

The workflow you will use

```
git clone <repo-url>    # once
git checkout -b feature/login-screen
git add .
git commit -m "Add login screen"
git push -u origin feature/login-screen

# open a Pull Request → review → merge

git checkout main && git pull
```

How the project is graded

The repository history is checked for:

- one branch per feature
- pull requests with peer review
- commits from every team member

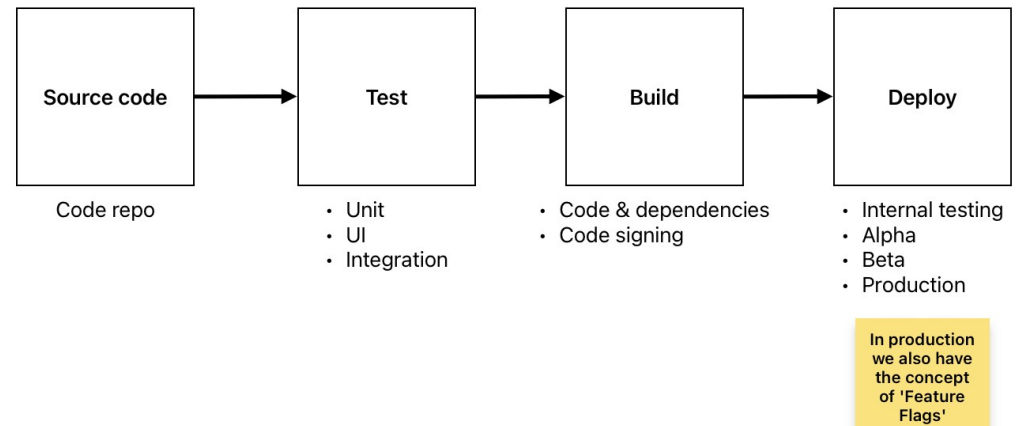
Pull requests, and what happens after merge

A pull request starts the process of merging your branch into the main branch

A peer reviews the changes and either approves them or requests changes

Until the author receives all required approvals, the PR stays open

After merge, a CI pipeline takes over: test → build → deploy



The CI pipeline: what runs after your pull request is merged

THE DEVICE LANDSCAPE

What is “mobile and embedded” computing?

The device landscape, and why it changes how you write software

One spectrum: from the cloud to a microcontroller

	 Cloud	 Laptop	 Phone	 Wearable	 SBC	 MCU
RAM	100s of GB	16–32 GB	8–16 GB	1–2 GB	1–8 GB	~0.5 MB
Storage	TB – PB	~1 TB	128–512 GB	~32 GB	SD card	~4 MB flash
Power	grid, kW	~60 W	~5 W, battery	~1 W, battery	~5 W	milliwatts
OS	Linux	macOS / Win	Android / iOS	watchOS / Wear	Linux	RTOS / none

This course lives on the right half of this table: resources are limited and the device runs on a battery.

What changes when you leave the desktop



Battery is a first-class resource

Every CPU cycle and radio wake-up costs energy. We measure this in Lecture 11



Thermal limits

Sustained load throttles the SoC, so extra CPU time is not always available



Intermittent connectivity

Offline is a normal state, not an error. Apps must be designed for it



You don't control deployment

App-store review, staged rollouts, and users who never update



Sensors & radios

GPS, accelerometer, camera and BLE: capabilities desktops don't have



ARM everywhere

From flagship phones down to microcontrollers: a different ISA than most servers

What every app negotiates for



CPU

compute



RAM

working memory



Storage

NAND flash



GPU

graphics & ML



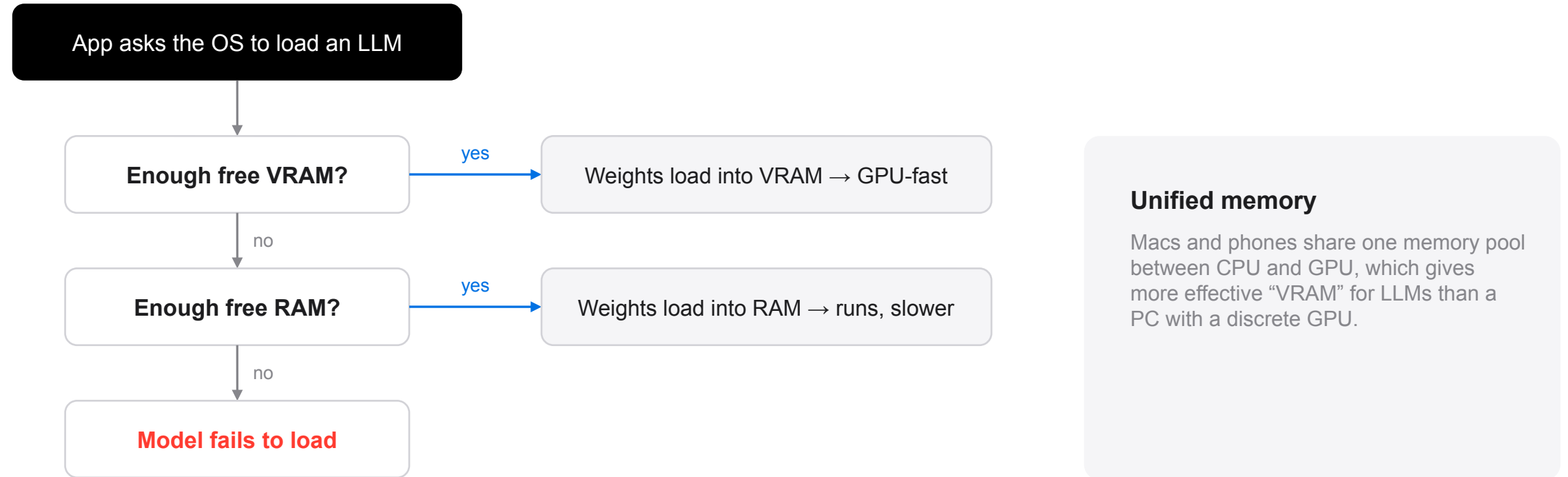
Battery

mobile-only: a finite budget

On a desktop, the first four are effectively elastic. On mobile, all five are hard budgets, **and the battery is the one your code cannot replenish.**

Next: one concrete resource negotiation, start to finish.

Resource allocation in action: loading an LLM



On-device AI is a resource-allocation problem. Keep this slide in mind for Lecture 9, AI on-device.

STRATEGIES

Four ways to build a mobile app

Web · Native · Cross-platform in two flavors, and how to choose

The baseline: native apps

Platform-specific: built for one OS, with that OS's own SDK and tools

Best possible performance, full access to every platform capability

Native UI components: the platform's own look and behavior

Works fully offline; distributed through Google Play / the App Store



iOS

Swift / SwiftUI · Xcode



Android

Kotlin / Jetpack Compose · Android Studio

Everything on the next slides is ultimately built on top of these two platform SDKs.

Strategy 1: web apps and PWAs

What you get

- + Opens from a URL: no installation, no store review
- + Updates reach every user instantly
- + One codebase, every platform with a browser
- + PWAs: installable, offline-capable, HTTPS-only

What you give up

- Limited sensor & hardware access: BLE, NFC, background GPS
- No reliable background execution, especially on iOS
- Browser performance ceiling, and WebKit-only on iOS
- No store presence: a different discoverability model

Web apps have their own set of architectures, SSR, SSG and CSR, which are a web-course topic. **Here we only care how the web option compares to the next three.**

Strategy 2: cross-platform, two architectures



React Native: drive native widgets

Your JavaScript / TypeScript code



JSI + Fabric, no serialized bridge since RN 0.76



Real native widgets on each OS

UI is genuinely native: per-platform look & feel
JS can update over the air (Expo EAS Update)
Every interaction crosses the JS ↔ native boundary



Flutter: draw every pixel

Your Dart code, AOT-compiled to native



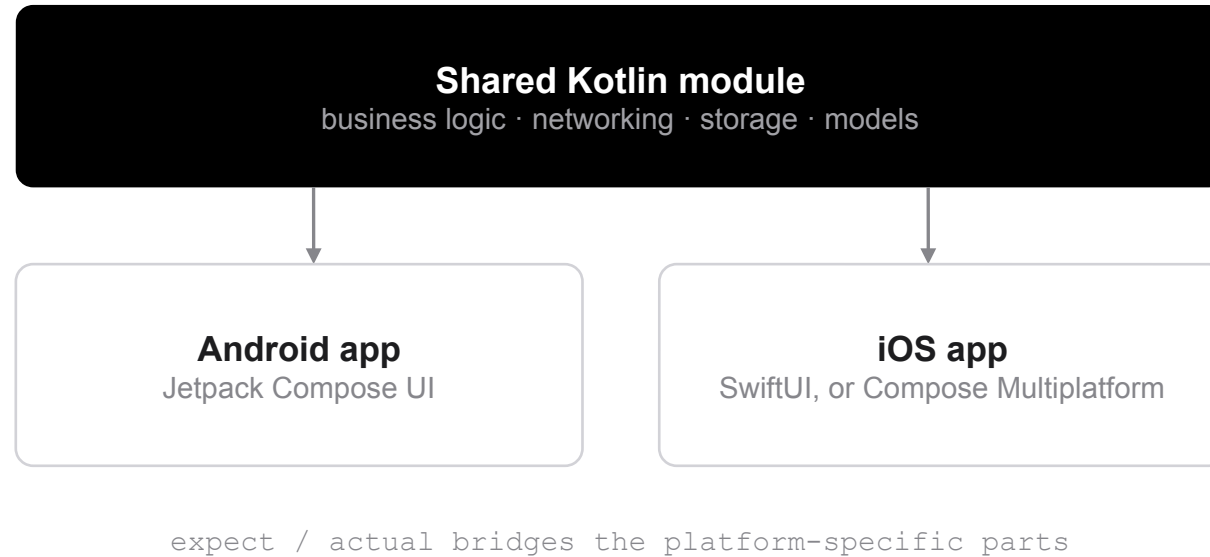
Impeller renderer: draws the entire UI



Identical UI on every platform

One rendering pipeline, so no per-platform differences
Talks to the OS via platform channels (→ Lecture 12)
Hot reload during development

Strategy 3: Kotlin Multiplatform, share the logic



Share as much, or as little, as you choose

Kotlin compiles natively per platform (Kotlin/Native for iOS)

Stable since Nov 2023; Compose Multiplatform for iOS stable since May 2025

In production at McDonald's, Netflix, Forbes

We build a shared KMP module in Lecture 12

Flutter shares everything, including the pixels. KMP shares exactly as much as you choose, and keeps each platform's native UI.

How to choose, and why this course uses Flutter

If you...

Reach without installs, light hardware needs

→ [Web / PWA](#)

Team lives in JS/React, wants native UI feel

→ [React Native](#)

Existing native apps, want shared logic

→ [Kotlin Multiplatform](#)

One team, pixel-identical UI, fast iteration

→ [Flutter](#)

Why Flutter, here

One codebase: a team of 3 can build the whole semester project

Hot reload: fast iteration in the labs

One renderer: what you see in the emulator is what you ship

Your instructor ships Flutter for a living: 45+ apps, 400k installs

Trade-offs

Bigger binaries than native apps

Platform-channel overhead when talking to the OS (→ Lecture 12)

Dart is knowledge you may not reuse outside Flutter

And where does “embedded” fit in?

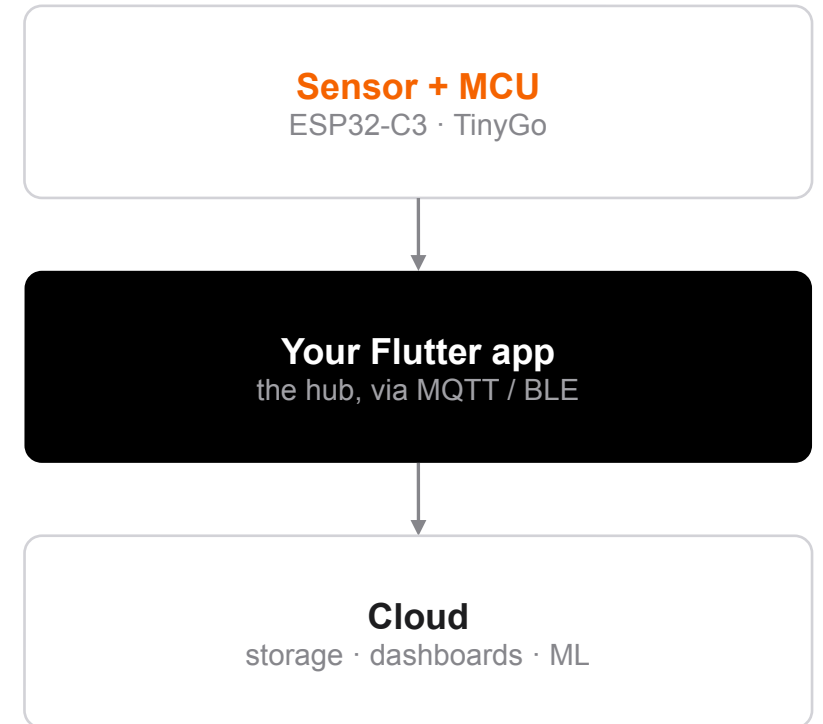
The same constraints, taken further: kilobytes of RAM, milliwatt power budgets, and often no operating system at all

The phone becomes the hub: nearby devices talk to your app over BLE or MQTT, and your app talks to the cloud

In Lecture 11: a TinyGo program on a microcontroller publishes sensor data, and your Flutter app consumes it live

Hardware note

TinyGo supports the ESP32-C3 / S3 (Wi-Fi support since TinyGo 0.41) and the Raspberry Pi Pico. Support for the original ESP32 is minimal.



The semester, pinned onto the spectrum



cloud



microcontroller

Cloud & backend

- L5 Serverless, VPS & Go backends
- L6 GraphQL, REST & observability
- L7 Auth, OAuth & App Check
- L8 WebSockets, permissions, routing

The phone

- L2 Dart, Kotlin & Flutter
- L3 Widgets & async
- L4 State management
- L9 AI on-device & LLMs
- L10 Offline-first & sync
- L12 Channels, FFI & KMP

Embedded

- L11 Performance & energy
- L11 TinyGo: sensors → MQTT → app

You'll see this map again at the start of every lecture, with a pin on where we are.

Before next week



Recap

One spectrum, cloud → microcontroller: this course covers the right half

Four strategies: web, native, cross-platform, KMP

We chose Flutter, with its trade-offs stated



This week

Create your GitHub account and open the course repo

Install the Flutter SDK. Lab 1 walks you through it

Start thinking about teammates and a project idea



Read more

flutter.dev/multi-platform

kotlinlang.org/multiplatform

tinygo.org

developer.android.com · developer.apple.com